

BitLab – A Minimal Bitcoin P2P Client in Python

Dawid Balikowski, Piotr Graczyk, Artur Czarnecki, Kacper Olszewski

January 26, 2026

Abstract

BitLab is a minimal Bitcoin peer-to-peer (P2P) client implemented in Python, designed for educational and experimental purposes. Unlike full Bitcoin nodes such as `bitcoind`, BitLab does not rely on external libraries or full blockchain validation. Instead, it focuses on demonstrating how Bitcoin nodes communicate using the native P2P wire protocol.

The project allows users to interact directly with real Bitcoin peers, perform the handshake, exchange messages such as `version`, `verack`, `ping`, `pong`, `getaddr`, `inv`, `getblocks`, `getheaders`, and even download raw blocks and transactions. Its primary goal is to make the Bitcoin networking layer transparent and understandable by exposing the protocol step by step.

Contents

1	Introduction	3
2	Bitcoin P2P Protocol Overview	4
3	Project Architecture	5
3.1	Main Components	5
3.2	BitcoinPeer	5
3.3	NonBlockingReader	6
4	Implemented Features	7
5	Encoding and Utility Functions	8
5.1	Double SHA-256	8
5.2	CompactSize (VarInt)	8
5.3	IPv4 to IPv6 Mapping	9

6	Example Workflow	9
6.1	Step 1: Starting the Client	9
6.2	Step 2: Connecting to the Network	9
6.3	Step 3: Discovering Peers	10
6.4	Step 4: Liveness Testing	10
6.5	Step 5: Header Synchronization	11
6.6	Step 6: Downloading Blocks	11
6.7	Step 7: Downloading Transactions	12
6.8	Educational Value	12
7	Conclusion	13

1 Introduction

Bitcoin is the first widely adopted decentralized digital currency system. Unlike traditional financial infrastructures, it operates without a central authority. Instead, it relies on a global peer-to-peer (P2P) network of nodes that exchange information about transactions and blocks using a custom binary protocol.

In practice, most developers interact with Bitcoin through high-level tools such as `bitcoind`, JSON-RPC APIs, or specialized libraries. These abstractions hide the internal mechanics of the network layer: message framing, handshakes, inventory exchange, block propagation, and peer discovery.

While this encapsulation is beneficial for production systems, it creates a barrier for learning. Understanding how Bitcoin nodes communicate at the wire level is essential for:

- grasping how decentralization is implemented in practice,
- analyzing network behavior and propagation delays,
- designing alternative clients or research tools,
- teaching distributed systems using a real-world protocol.

BitLab was created to remove this barrier. It is a minimal Bitcoin P2P client written entirely in Python, without relying on `bitcoind` or external Bitcoin libraries. Every message is constructed manually, every byte on the wire is visible, and every protocol step is explicit.

The project does not aim to replace a full node. It does not validate blocks, execute scripts, or maintain a blockchain state. Instead, it focuses on a single goal: making the Bitcoin P2P protocol understandable through direct experimentation.

By providing an interactive REPL and a compact codebase, BitLab allows the user to:

- connect to real Bitcoin peers,
- perform the native handshake,
- inspect raw protocol messages,
- request headers, blocks, and transactions,
- compute hashes exactly as Bitcoin nodes do.

In this sense, BitLab functions as a laboratory for the Bitcoin network, bridging the gap between protocol documentation and real-world behavior.

2 Bitcoin P2P Protocol Overview

Communication between Bitcoin nodes is based on a custom binary protocol operating over TCP. All messages share a common framing format that allows nodes to identify message boundaries and verify integrity.

Each P2P message has the following structure:

Magic (4B)	Command (12B)	Length (4B)	Checksum (4B)
Payload (variable length)			

- **Magic** identifies the network (e.g., mainnet or testnet).
- **Command** is an ASCII string such as "**version**" or "**ping**", padded with null bytes to 12 bytes.
- **Length** specifies the size of the payload in bytes.
- **Checksum** is the first four bytes of SHA256(SHA256(payload)).
- **Payload** contains message-specific data.

This framing allows nodes to process a continuous byte stream and reliably extract individual messages, even when they arrive fragmented or concatenated.

Before any meaningful communication can occur, two nodes must perform a handshake. This process consists of:

1. Sending a **version** message describing protocol capabilities,
2. Receiving the peer's **version**,
3. Exchanging **verack** acknowledgements.

Only after this exchange is the connection considered valid.

Subsequent communication is based on explicit message types, including:

- **ping/pong** for liveness checks,
- **getaddr/addr** for peer discovery,
- **inv** for advertising objects,
- **getdata** for requesting blocks or transactions,
- **getheaders/headers** for chain synchronization,
- **block** and **tx** for raw data transfer.

Every object in the network is referenced by its hash, and all transfers are inventory-driven: a node never sends data unless explicitly requested.

BitLab implements this protocol directly. It constructs the message header by hand, computes checksums using double SHA-256, encodes integers using Bitcoin's CompactSize format, and parses incoming byte streams without relying on external abstractions.

As a result, the project exposes the Bitcoin P2P protocol exactly as it exists on the wire, turning each network exchange into an observable and reproducible experiment.

3 Project Architecture

The BitLab project is structured around a small number of core components that together implement a minimal yet functional Bitcoin P2P client. The architecture emphasizes clarity and direct correspondence with the Bitcoin wire protocol.

3.1 Main Components

The system is built from the following elements:

- `BitcoinPeer` – manages a single connection to a Bitcoin node,
- `NonBlockingReader` – buffers socket data and extracts complete P2P messages,
- Message-building and parsing utilities,
- A simple interactive REPL loop.

3.2 BitcoinPeer

The `BitcoinPeer` class represents a single remote Bitcoin node. It is responsible for:

- Establishing TCP connections,
- Performing the P2P handshake,
- Sending and receiving protocol messages,
- Maintaining session state.

The handshake logic is implemented as follows:

Listing 1: Handshake procedure in `BitcoinPeer`

```
def handshake(self):
    print("[+] Sending 'version'...")
    self._send_msg("version", build_version_payload(self.ip, self.port))
```

```

got_version = False
got_verack = False
end = time.time() + 20.0

while time.time() < end and not (got_version and got_verack):
    msgs = self.reader.poll_messages(end - time.time())
    for cmd, pl in msgs:
        if cmd == "version":
            got_version = True
            self._send_msg("verack", b"")
        elif cmd == "verack":
            got_verack = True
        elif cmd == "ping":
            self._send_msg("pong", pl)

if not (got_version and got_verack):
    raise TimeoutError("Handshake_failed.")

```

This function mirrors the behavior of a real Bitcoin node:

1. The client sends a **version** message,
2. Waits for the peer's **version**,
3. Responds with **verack**,
4. Waits for the peer's **verack**.

Only after this exchange is the connection considered valid.

3.3 NonBlockingReader

Network I/O is handled asynchronously using `select`. The `NonBlockingReader` class accumulates bytes and extracts full P2P messages:

Listing 2: Message extraction in `NonBlockingReader`

```

magic, cmd_raw, length, csum = struct.unpack("<4s12sI4s", self.buf[:24])
if magic != MAINNET_MAGIC:
    raise ValueError("Invalid_magic_bytes")

total_len = 24 + length
payload = bytes(self.buf[24:total_len])
cmd = cmd_raw.rstrip(b"\x00").decode("ascii")

```

This implementation directly reflects the Bitcoin P2P framing format and makes the protocol transparent.

4 Implemented Features

BitLab implements a meaningful subset of the Bitcoin P2P protocol:

- Interactive REPL console,
- DNS seed bootstrapping,
- Full handshake (`version` / `verack`),
- Liveness testing with `ping` / `pong`,
- Peer discovery via `getaddr` and `addr`,
- Inventory exchange (`inv`, `getdata`),
- Block header synchronization (`getheaders`),
- Block and transaction download,
- Correct TXID computation, including SegWit handling.

For example, sending a `ping` message is implemented as:

Listing 3: Sending a ping message

```
def build_ping_payload() -> bytes:
    nonce = random.getrandbits(64)
    return struct.pack("<Q", nonce)
```

The payload is then wrapped into a full P2P message:

Listing 4: P2P message construction

```
def build_message(command: str, payload: bytes) -> bytes:
    cmd = command.encode("ascii")
    cmd_padded = cmd + b"\x00" * (12 - len(cmd))
    length = struct.pack("<I", len(payload))
    csum = checksum(payload)
    return MAINNET_MAGIC + cmd_padded + length + csum + payload
```

This demonstrates how BitLab explicitly constructs every network message without abstraction layers.

5 Encoding and Utility Functions

The project reimplements core Bitcoin primitives that are usually hidden inside full-node software.

5.1 Double SHA-256

Bitcoin uses a double SHA-256 hash in many places:

Listing 5: Double SHA-256

```
def sha256d(data: bytes) -> bytes:
    return hashlib.sha256(hashlib.sha256(data).digest()).digest()
```

This function is used for:

- Computing block hashes,
- Computing transaction IDs,
- Generating message checksums.

5.2 CompactSize (VarInt)

Bitcoin encodes variable-length integers using a special format:

Listing 6: CompactSize encoding

```
def compact_size_encode(n: int) -> bytes:
    if n < 0xfd:
        return struct.pack("<B", n)
    elif n <= 0xffff:
        return b"\xfd" + struct.pack("<H", n)
    elif n <= 0xffffffff:
        return b"\xfe" + struct.pack("<I", n)
    else:
        return b"\xff" + struct.pack("<Q", n)
```

This encoding is used for:

- Vector lengths,
- Script sizes,
- Transaction input/output counts.

5.3 IPv4 to IPv6 Mapping

Bitcoin uses IPv6 fields even for IPv4 peers:

Listing 7: IPv4 to IPv6-mapped conversion

```
def ipv6_from_ipv4(ipv4_str: str) -> bytes:
    parts = bytes(int(x) for x in ipv4_str.split("."))
    return b"\x00" * 10 + b"\xff\xff" + parts
```

This ensures compatibility with the protocol format and mirrors real node behavior.

Together, these utilities make the low-level Bitcoin protocol explicit and educational, allowing the reader to see exactly how data is represented on the wire.

6 Example Workflow

A typical BitLab session follows a clear, educational workflow that reflects how a real Bitcoin node communicates with the network. Each step corresponds directly to an operation in the Bitcoin P2P protocol and exposes its internal mechanics.

6.1 Step 1: Starting the Client

After launching the program, the user is presented with a simple REPL prompt:

```
=== BitLab - start ===
Type 'help' to list commands.
bitlab>
```

At this point, no network connection exists. All operations that require a peer will fail until a connection is established.

6.2 Step 2: Connecting to the Network

The recommended entry point is:

```
bitlab> connectseed
```

This command:

1. Queries public DNS seed servers,
2. Resolves them into IPv4 addresses,
3. Randomly selects candidate peers,

4. Attempts TCP connections,
5. Performs the full P2P handshake.

Internally, once a socket is established, BitLab sends a **version** message and waits for:

- **version** from the peer,
- **verack** confirming protocol compatibility.

Only after this exchange does the program consider the connection valid.

6.3 Step 3: Discovering Peers

With an active connection, the user may request additional peers:

```
bitlab> getaddr
```

This sends a **getaddr** message and waits for **addr**. The received payload is parsed into tuples:

(IP, Port, Timestamp, Services)

The user can later display them with:

```
bitlab> peers
```

This step demonstrates how Bitcoin nodes bootstrap the network without central coordination.

6.4 Step 4: Liveness Testing

To verify that the remote node is responsive:

```
bitlab> ping
```

BitLab sends a random 64-bit nonce in a **ping** message and waits for a matching **pong**. This round-trip mirrors the behavior of production nodes and confirms that the connection is alive.

6.5 Step 5: Header Synchronization

Using a known block hash, the user may query block headers:

```
bitlab> getheaders <block_hash>
```

This sends a `getheaders` message containing:

- Protocol version,
- A vector of locator hashes,
- A stop hash.

The peer replies with `headers`, each consisting of:

- 80-byte block header,
- A transaction count placeholder.

BitLab parses these headers, computes their hashes, and displays:

- Block hash,
- Previous block hash,
- Timestamp,
- Difficulty bits.

This step shows how nodes synchronize blockchain structure without downloading full blocks.

6.6 Step 6: Downloading Blocks

To retrieve a full block:

```
bitlab> getblock <block_hash>
```

The client:

1. Sends `getdata` with an inventory entry of type `BLOCK`,
2. Waits for a `block` message,
3. Parses the 80-byte header,
4. Reads the transaction count,

5. Iterates through transactions.

For each transaction, BitLab reconstructs the legacy serialization (excluding witness data when present) and computes the correct TXID using:

$$\text{TXID} = \text{SHA256}(\text{SHA256}(\text{legacy_bytes}))$$

The first transaction IDs are stored and displayed.

6.7 Step 7: Downloading Transactions

Using one of the extracted TXIDs:

```
bitlab> gettx <txid>
```

or

```
bitlab> gettxi <n>
```

BitLab requests the transaction via `getdata(TX)` and waits for a `tx` message. The raw payload is returned and hashed locally, allowing the user to verify correctness.

6.8 Educational Value

This workflow exposes the real operational sequence of a Bitcoin node:

- Bootstrapping through DNS seeds,
- Handshake and protocol negotiation,
- Peer discovery,
- Inventory-based object exchange,
- Header-first synchronization,
- On-demand retrieval of blocks and transactions.

Unlike full nodes, BitLab makes every step visible and inspectable, turning the Bitcoin network into an interactive laboratory.

7 Conclusion

BitLab demonstrates that the Bitcoin peer-to-peer protocol, often perceived as complex and opaque, can be understood with a relatively small and readable codebase. By stripping away the layers of abstraction present in full-node implementations, the project exposes the raw mechanics of decentralized communication.

Through direct interaction with real Bitcoin peers, the user observes how:

- nodes discover each other without central coordination,
- protocol compatibility is negotiated via the handshake,
- objects are advertised using inventory messages,
- blocks and transactions are transferred on demand,
- cryptographic hashes define identity and integrity.

Instead of treating the Bitcoin network as a black box, BitLab turns it into a transparent system in which every byte on the wire has meaning. This approach bridges the gap between formal protocol specifications and real-world behavior.

From an educational perspective, BitLab serves as a practical laboratory for distributed systems. It allows students to experiment with a production-grade protocol, observe network dynamics, and understand how decentralization is implemented in practice. The project also highlights that many core concepts of Bitcoin—message framing, peer discovery, inventory-driven synchronization, and cryptographic verification—are independent of block validation or consensus logic.

Although BitLab is not suitable for production use and intentionally omits security-critical features, its value lies in clarity rather than completeness. It proves that meaningful interaction with the Bitcoin network does not require a full node, and that even a minimal client can reveal the essential structure of one of the world’s most influential distributed systems.

In this sense, BitLab is not merely a tool, but a pedagogical framework: it transforms the Bitcoin network from an abstract concept into an observable, interactive environment, making the principles of peer-to-peer systems tangible and accessible.